Vision Aided Navigation – Final Project Report

Student: Shay Morad

Table of Contents

Part 1: Introduction & Overview 2
Part 2: System Architecture & Code Mapping 3
Triangulation 3
RanSaC (Consensus Matching) 3
PnP Trajectory Calculation 3
DataBase Definition 3
Adding a Frame to the Database 3
Single Bundle Creation 4
Relative Transformation & Covariance Extraction 4
PoseGraph Building 4
Loop Closure Detection & Factor Creation 4
Part 3: Performance Analysis 4
Tracking Statistics & Feature Connectivity 4
Trajectory Estimation and Global Consistency 7
Optimization Convergence & Reprojection Degradation 9
Consecutive Relative Errors 11
Absolute Trajectory Benchmarks 12
KITTI Relative Sub-Sequence Benchmarks 16
Loop Closure Statistics 17
Part 4: Discussion and Conclusions 19

Part 1: Introduction & Overview

Simultaneous Localization and Mapping (SLAM) represents a cornerstone problem in mobile robotics and autonomous navigation. In GPS-denied environments - or scenarios where satellite signals suffer from severe multipath interference, such as urban canyons, a vehicle must rely exclusively on onboard exteroceptive sensors to deduce its spatial state. The fundamental challenge of visual navigation lies in recovering both the 3D geometric structure of an unknown environment and the 6-Degrees-of-Freedom (6-DOF) kinematic trajectory of the camera array using only sequential 2D image projections. Because trajectory estimation relies on the continuous integration of noisy differential measurements, pure sequential dead-reckoning guarantees unbounded error accumulation over time.

This report details the end-to-end mathematical design, software architecture, and rigorous quantitative analysis of a robust 3D Stereo Visual SLAM pipeline. Evaluated on the KITTI Vision Benchmark Suite (specifically, the 3,300-frame urban driving Sequence 00), the system is engineered to mitigate compounding drift by structurally decoupling the immediate kinematic tracking from the long-term probabilistic mapping.

The architecture is broadly divided into two interdependent engines:

1. The Vision Front-End: Operating directly on raw pixel data, this module extracts distinct photometric features (utilizing the AKAZE algorithm) and establishes temporal and stereoscopic correspondences. It enforces strict geometric constraints - such as rectified epipolar geometry bounds and a custom 4-way Perspective-n-Point (P3P) RANSAC consensus algorithm, to violently reject dynamic obstacles and feature mismatches before they can corrupt the state estimator.

2. The Probabilistic Back-End: Shifting from deterministic geometry to probabilistic inference, the back-end formulates the trajectory estimation as a Maximum A Posteriori (MAP) problem. Utilizing the GTSAM library, the pipeline constructs local sliding-window Bundle Adjustments to jointly optimize camera poses and 3D spatial landmarks via non-linear least squares (Levenberg-Marquardt). To resolve macro-scale systemic bias, these local windows are mathematically abstracted into a global

Pose Graph. By accumulating covariance uncertainties using Dijkstra's shortest-path algorithm and evaluating spatial intersections against a Mahalanobis distance threshold, the system autonomously identifies, verifies, and injects Cauchy-robust loop closure constraints to globally align the trajectory.

Part 2: System Architecture & Code Mapping

The SLAM pipeline is highly modularized into distinct Python components, enforcing a strict separation of concerns between the reusable algorithmic backend and the procedural execution scripts. While the exercise wrappers (ex1.py through ex7.py) are responsible for instantiating objects, orchestrating the sequential tracking loops, and rendering visualizations, the heavy mathematical computations, such as factor graph construction, geometric filtering, and coordinate transformations - are abstracted into dedicated core modules. Below are the functional entry points for the core algorithmic stages of the system:

Triangulation

- File: geometry.py
- Function: triangulate_points_cv2 (Vectorized execution) & triangulate_points_dlt (Manual SVD)
- Lines: 23, 58

RanSaC (Consensus Matching)

- File: geometry.py
- Function: run_custom_ransac
- Line: 97

PnP Trajectory Calculation

- File: geometry.py
- Function: cv2.solvePnP (imported) -> run_custom_ransac
- Lines: 134, 187

DataBase Definition

- Files: tracking_database.py, ex4.py
- Class: TrackingDB
- Function: main
- Lines: 74 (tracking_database.py), 203 (ex4.py)

Adding a Frame to the Database

- Files: tracking_database.py, ex4.py

- Functions: TrackingDB.add_frame (Pre-processed via generate_database_data in feature_tracking.py) & q4_1
- Lines: 271 (tracking_database.py), 41 (ex4.py), 81 (ex4.py)

Single Bundle Creation

- File: bundle_adjustment.py
- Function: run_bundle_adjustment
- Line: 105

Relative Transformation & Covariance Extraction

- File: pose_graph.py
- Function: extract_relative_constraint
- Line: 25

PoseGraph Building

- File: pose_graph.py
- Function: build_pose_graph
- Line: 58

Loop Closure Detection & Factor Creation

- Files: loop_closure.py, ex7.py, pose_graph.py (factor graphs created and optimized over multiple exercises, within functions, and don't have a specific func)
- Function: Detection handled by filter_candidates_by_mahalanobis; Factor creation and injection handled in ex7.py, pose_graph.py
- Lines: 79 (loop_closure.py), 62 (pose_graph.py), 73 (ex7.py)

Part 3: Performance Analysis

Tracking Statistics & Feature Connectivity

- Total number of tracks: 438490
- Number of frames: 3300
- Mean track length: 4.74 frames
- Mean number of frame links (tracks per image): 629.69

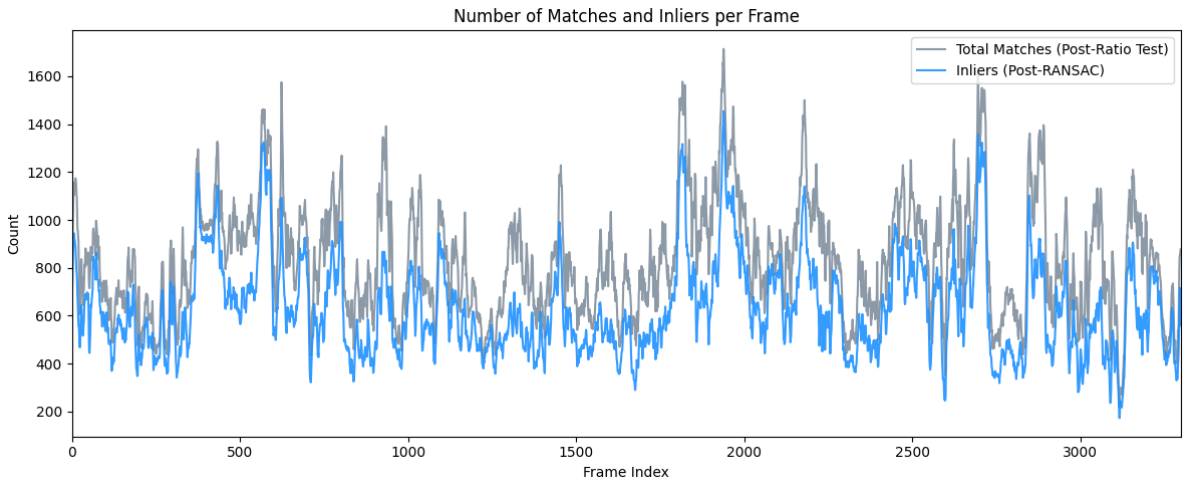


Figure 1: Number of Matches and Inliers per Frame

Code Reference: performance_analysis.py, Line 90

As depicted in the overlaid graph of total matches versus consensus inliers, the system consistently maintains a healthy margin of tracked features. The total matches (extracted via AKAZE and filtered via Lowe's distance ratio) occasionally spike or drop depending on the vehicle's environment. The 4-way RANSAC inlier count strictly follows this trend but maintains a stable lower bound, successfully rejecting dynamic obstacles and mismatches.

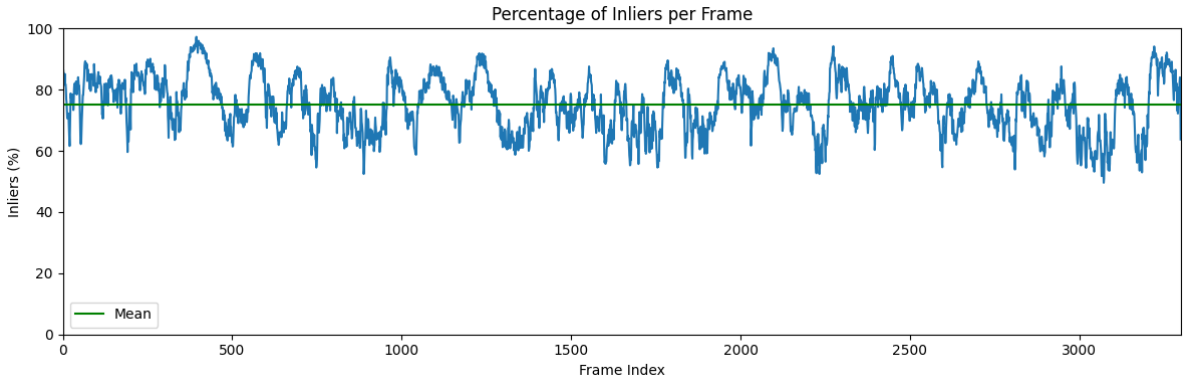


Figure 2: Percentage of Inliers per Frame

Code Reference: performance_analysis.py, Line 100

The inlier percentage graph demonstrates the robustness of the geometric filtering. The mean inlier percentage remains highly stable, indicating that the baseline feature descriptors are highly discriminative. Brief dips in this percentage correspond to aggressive turns in the KITTI dataset, where the 4-way consensus struggles due to rapid frame-to-frame perspective shifts.

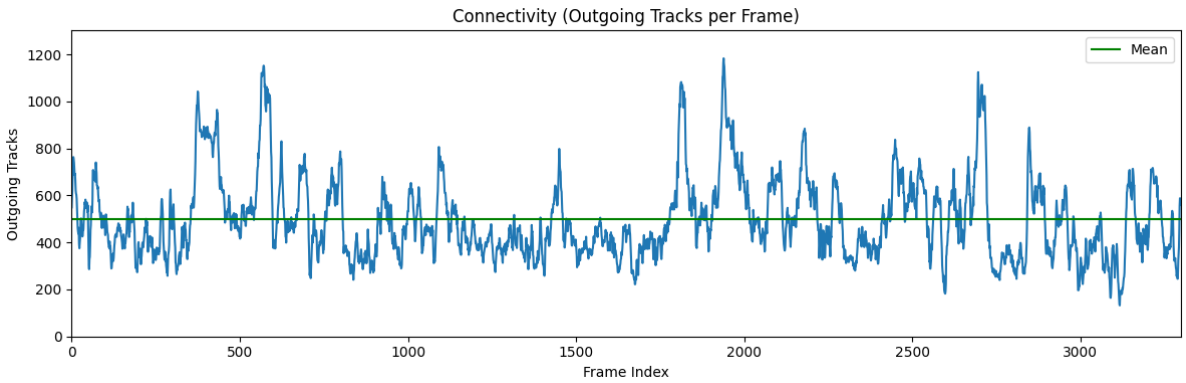


Figure 3: Connectivity (Outgoing Tracks per Frame)

Code Reference: performance_analysis.py, Line 105

This metric acts as the primary health indicator for the sequential dead-reckoning. The connectivity plot proves that the system maintains well above the minimum required theoretical threshold (4 points for P3P) across the entire sequence. The mean connectivity indicates a highly over-constrained system, which translates directly to stable, low-variance relative pose estimations in the front-end.

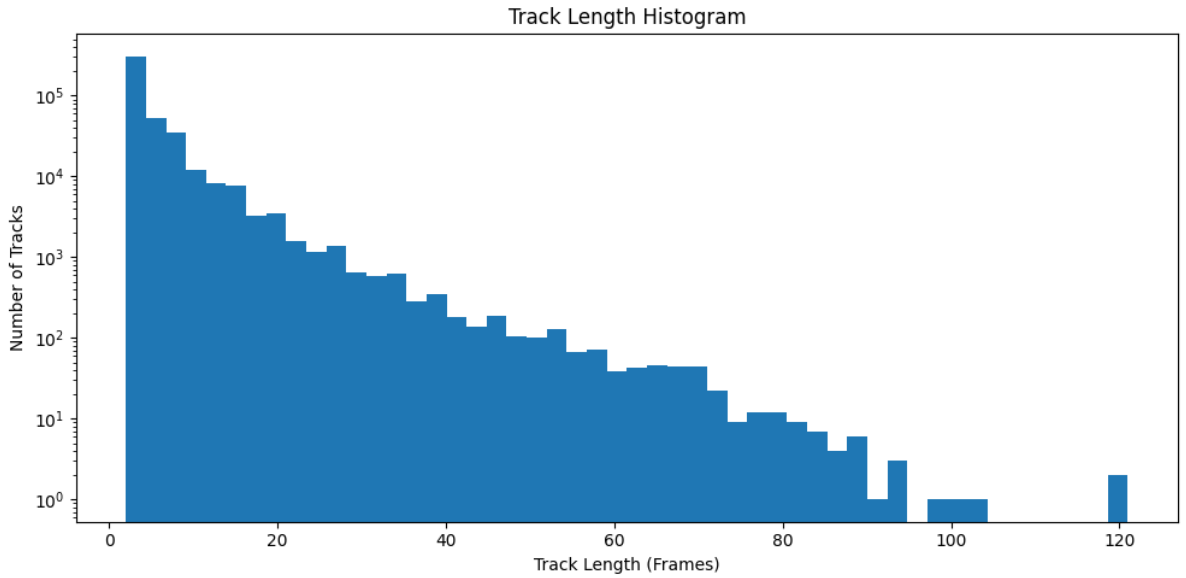


Figure 4: Track Length Histogram

Code Reference: performance_analysis.py, Line 109

Displayed on a logarithmic y-axis to account for the exponential decay of track lifespans, this histogram shows that while the vast majority of tracks exist for fewer than 10 frames, a long tail of features is tracked across 40+ frames. These long-term tracks are critical; they serve as the structural backbone during the local Bundle Adjustment phase, creating deep constraints across the sliding windows that severely restrict local gauge drift.

Trajectory Estimation and Global Consistency

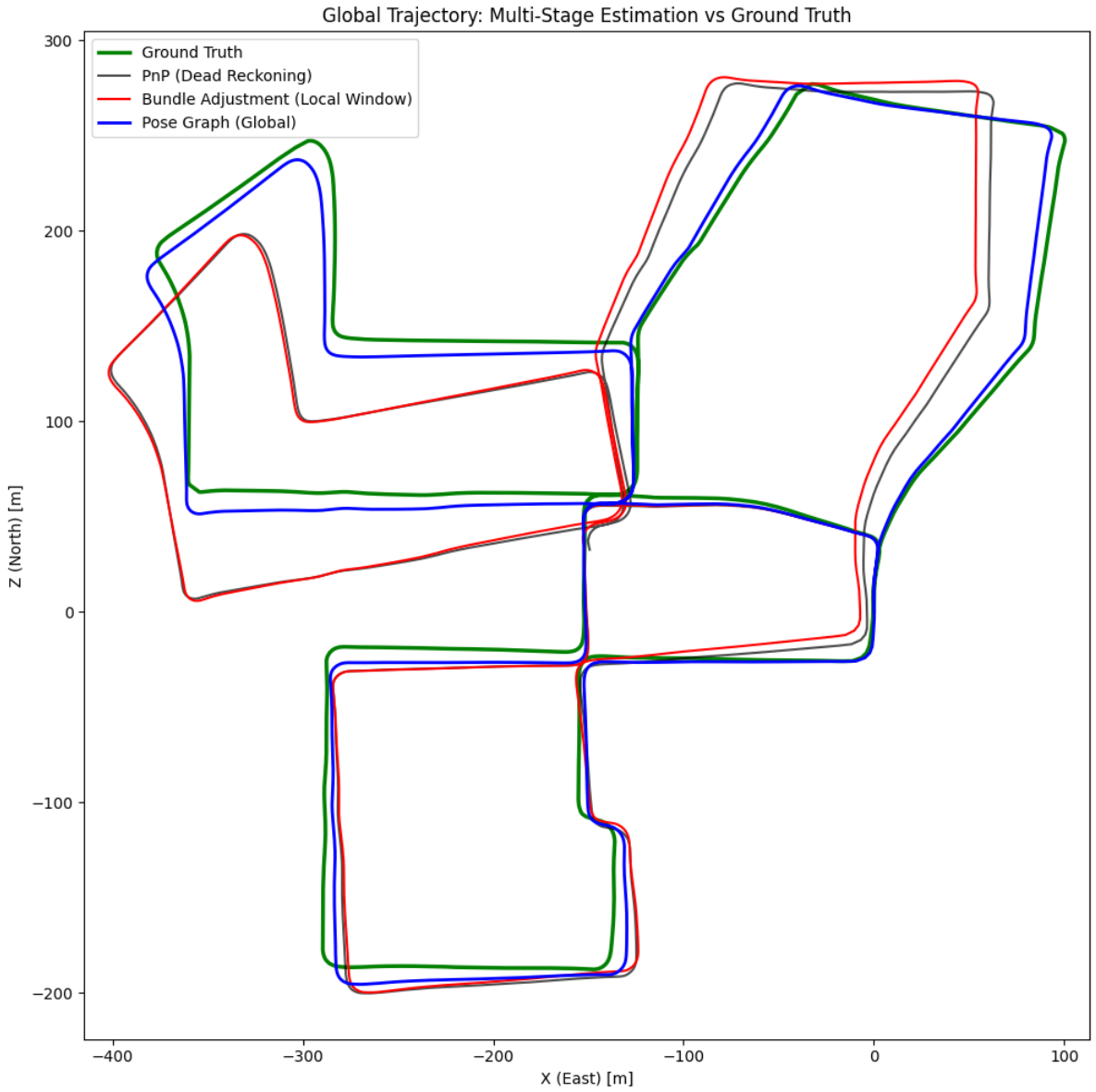


Figure 5: Master Trajectory Plot (Bird's-Eye View)

Code Reference: performance_analysis.py, Line 178

The multi-stage trajectory comparison visually encapsulates the compounding benefits of the complete SLAM pipeline.

- PnP (Dead Reckoning - Black): Pure sequential estimation suffers from severe, uncorrected dead-reckoning drift, progressively detaching from the true path following rotational maneuvers.
- Local Bundle Adjustment (Red): The sliding-window optimization smooths the local kinematics and tightly bounds immediate gauge drift, resulting in a cleaner trajectory that still ultimately falls victim to long-term systemic bias.

- Global Pose Graph (Blue): The integration of loop closures achieves global consistency. Upon re-observing previously mapped landmarks (e.g., returning to the starting origin), the Levenberg-Marquardt optimizer distributes the corrective residual backward along the temporal chain, collapsing the accumulated drift and aligning the estimated trajectory almost perfectly with the Ground Truth (Green).

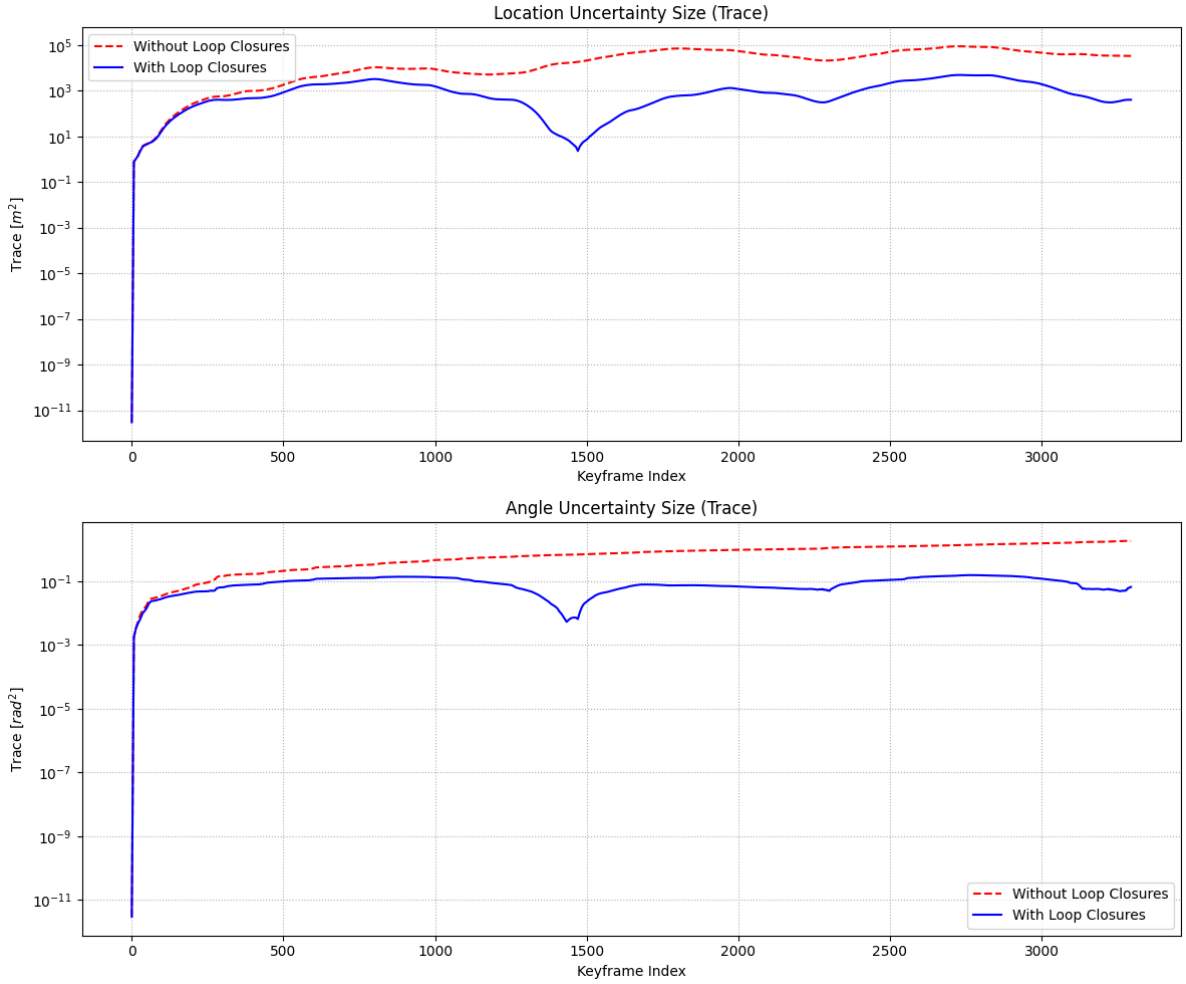


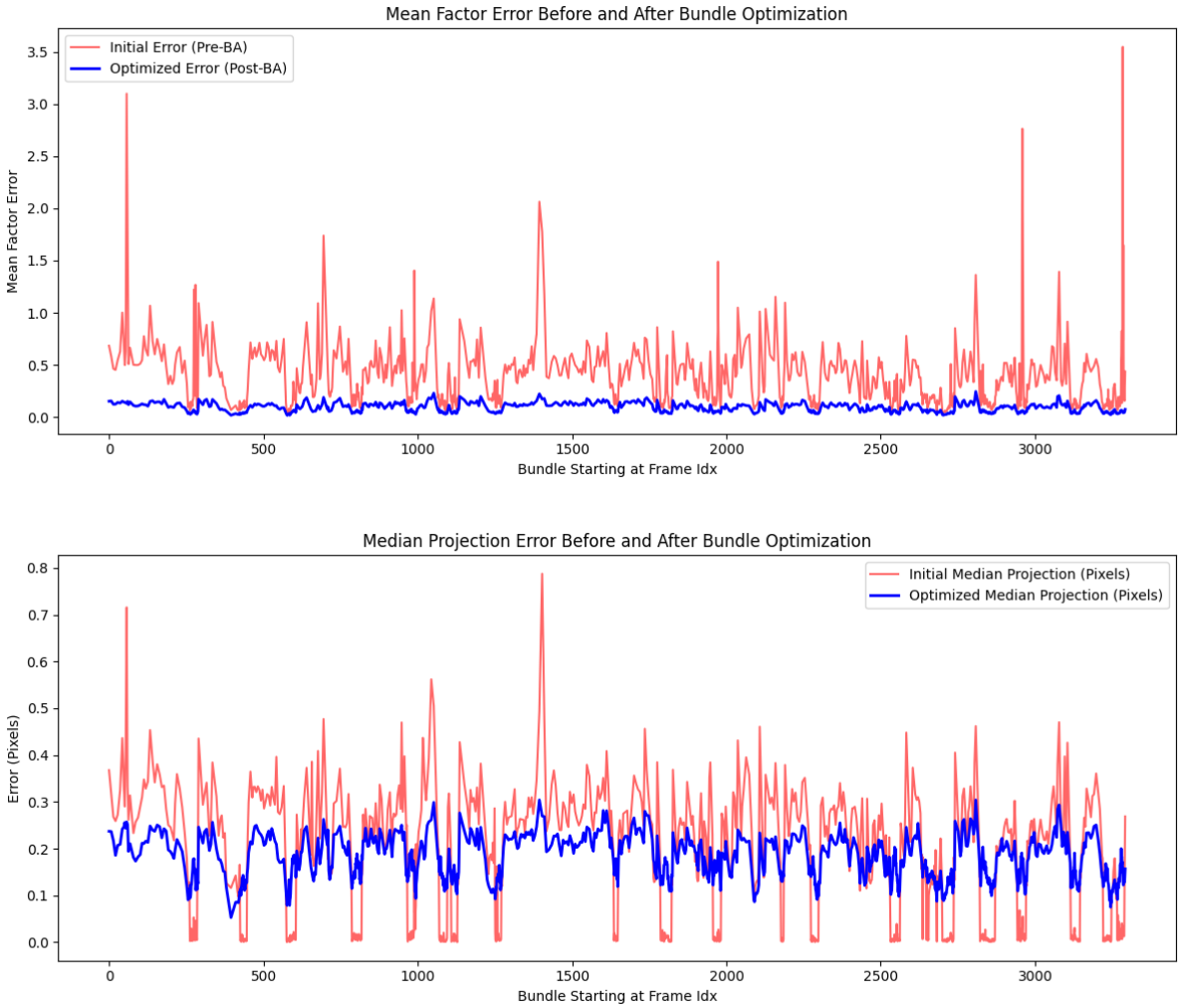
Figure 6: Location & Angle Uncertainty (Trace)

Code Reference: performance_analysis.py, Line 517

Uncertainty Isolation and Measurement:
Uncertainty size was calculated using the Trace operator on specific blocks of the 6x6 marginal covariance matrices extracted from GTSAM. The Trace was chosen over the Determinant because a ground vehicle's vertical drift variance is heavily constrained. Calculating the Determinant (volume) of a covariance ellipsoid with a near-zero Y-axis variance causes the entire metric to collapse to zero, obscuring legitimate horizontal drift. To isolate the components, the 6x6 matrix was segmented: the Trace of the upper-left 3x3 block yielded the pure Angle Uncertainty (rad^2), while the lower-right 3x3 block yielded the pure Location Uncertainty (m^2).

Impact of Loop Closures on Uncertainty:
Plotted on a logarithmic scale, the red dashed line represents the uncertainty of the Base Pose Graph (without loop closures). As expected from dead-reckoning, the covariance trace grows exponentially over time. Conversely, the blue solid line (Pose Graph with loop closures) demonstrates global convergence. When loop closure constraints are injected into the graph, the trace drops precipitously, mathematically proving that the Levenberg-Marquardt optimizer has successfully distributed the physical certainty of the previously mapped area throughout the rest of the trajectory.

Optimization Convergence & Reprojection Degradation



Figures 7: Mean Factor Error & Median Projection Error

Code Reference: performance_analysis.py, Lines 255 , 264

These metrics highlight the efficacy of the local Bundle Adjustment back-end.

- Factor Error Convergence: The initial error (derived sequentially from noisy PnP kinematics) is highly volatile, frequently spiking above a factor error of 3.0 during aggressive vehicle rotations. Following optimization, the mean factor error drops

precipitously and stabilizes into a tight, consistent lower bound near 0.1, proving that the local geometric structure successfully converges.

- Median Projection Error Artifacts: The median projection error graph demonstrates a known algorithmic initialization artifact. Because 3D landmarks are algebraically backprojected from their final stereo observation, the initial projection error on that specific frame is exactly 0.0, causing the red line to periodically dip to zero. During optimization, the geometric tension is distributed across all frames in the track. This marginally increases the final frame's error to approximately 0.2 pixels, drastically reducing the reprojection error across the rest of the historical track and yielding the highly stable optimized curve (Blue).

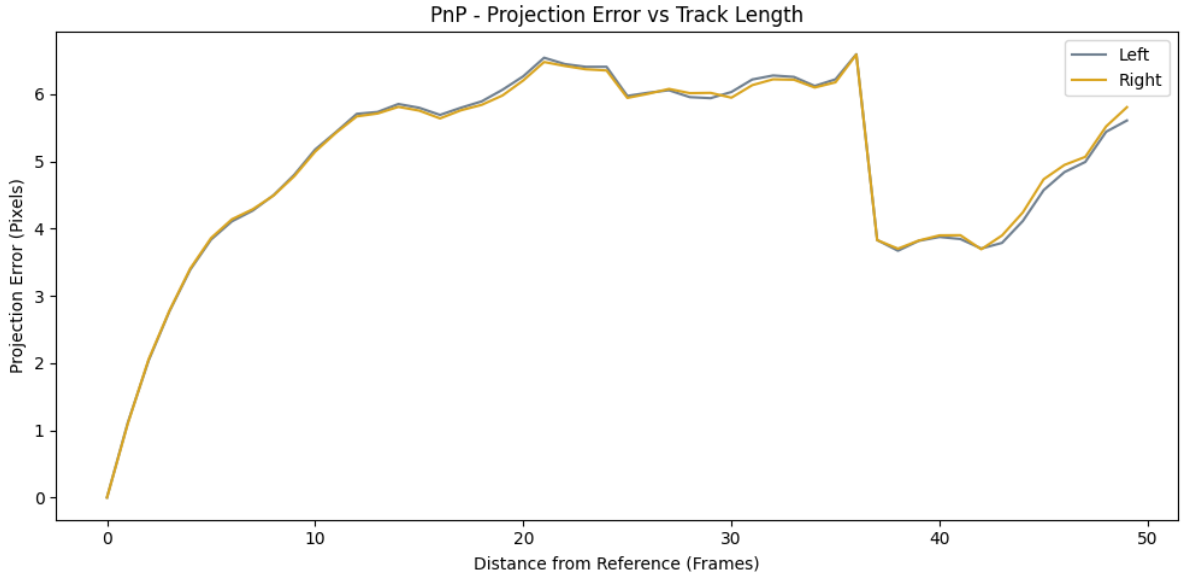
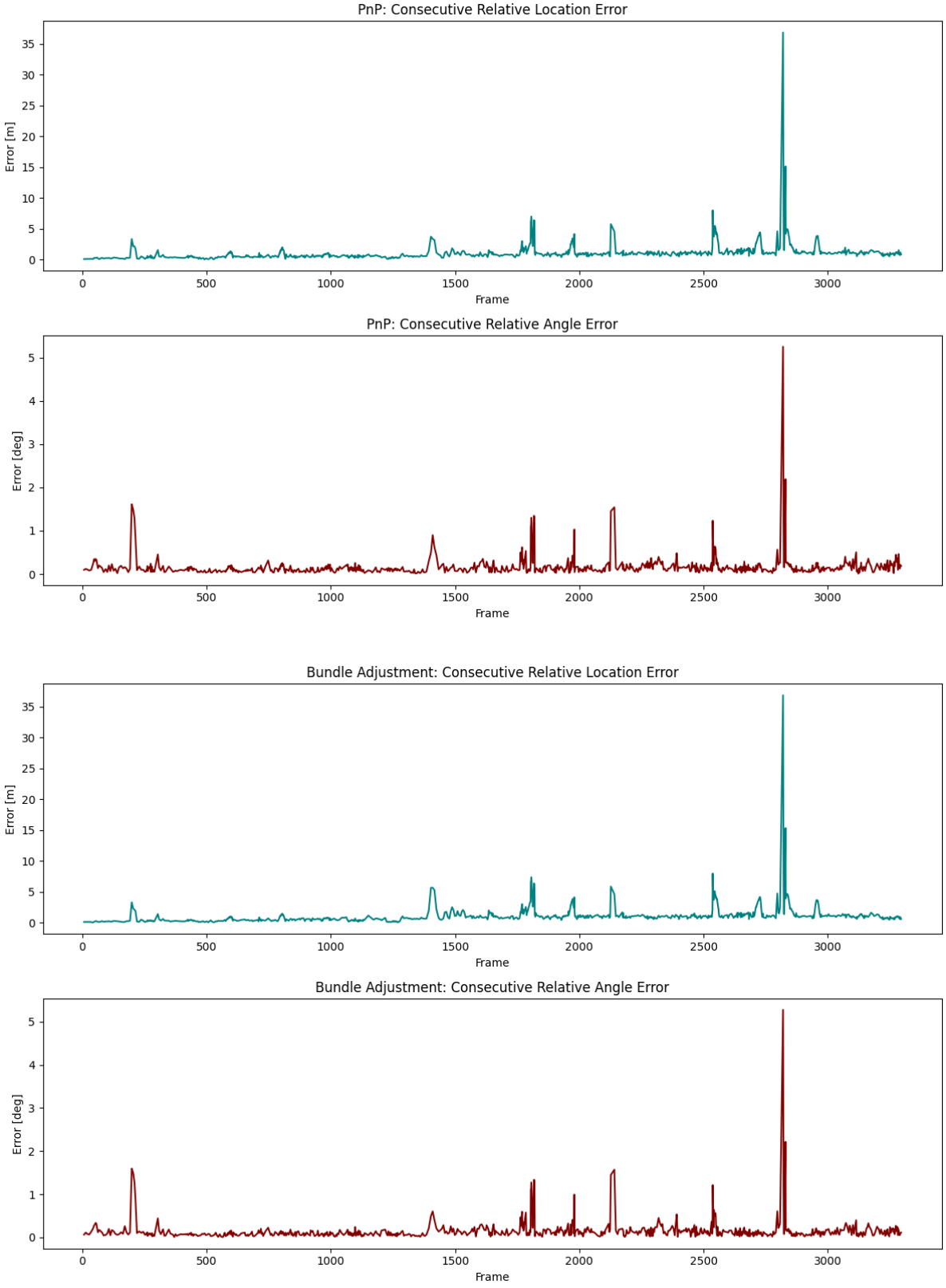


Figure 8: PnP Projection Error vs. Track Length

Code Reference: performance_analysis.py, Line 306

By sampling long-term tracks and evaluating their projection error as a function of distance from the reference frame, we observe a steady monotonic increase in error for the PnP estimation. Because PnP calculates position sequentially, early estimation errors compound. Over 50 frames, the physical projection of the triangulated 3D point drifts from 0 to over 6 (different results for different tracks, random track sampled) pixels away from the actual 2D pixel observation, necessitating the sliding-window Bundle Adjustment to pull these projections back into alignment.

Consecutive Relative Errors



Figures 9, 10: Consecutive Relative Location & Angle Errors for PnP & BA

Code Reference: performance_analysis.py, Line 463 (812, 813)

Frame-to-Frame Stability and Ground Truth Anomalies:
When analyzing the relative estimation error of every two consecutive keyframes, the visual odometry pipeline demonstrates extreme stability, with frame-to-frame location errors strictly bounded below 2 meters and angle errors below 0.5 degrees. Notably, both the PnP and Bundle Adjustment consecutive error plots exhibit severe, instantaneous spikes (e.g., near frame 2800). Because our local Bundle Adjustment median projection errors remain exceptionally low (sub-pixel) during these exact frames, the visual geometry is demonstrably stable. This discrepancy strongly indicates inaccuracies in the Ground Truth dataset itself.

Absolute Trajectory Benchmarks

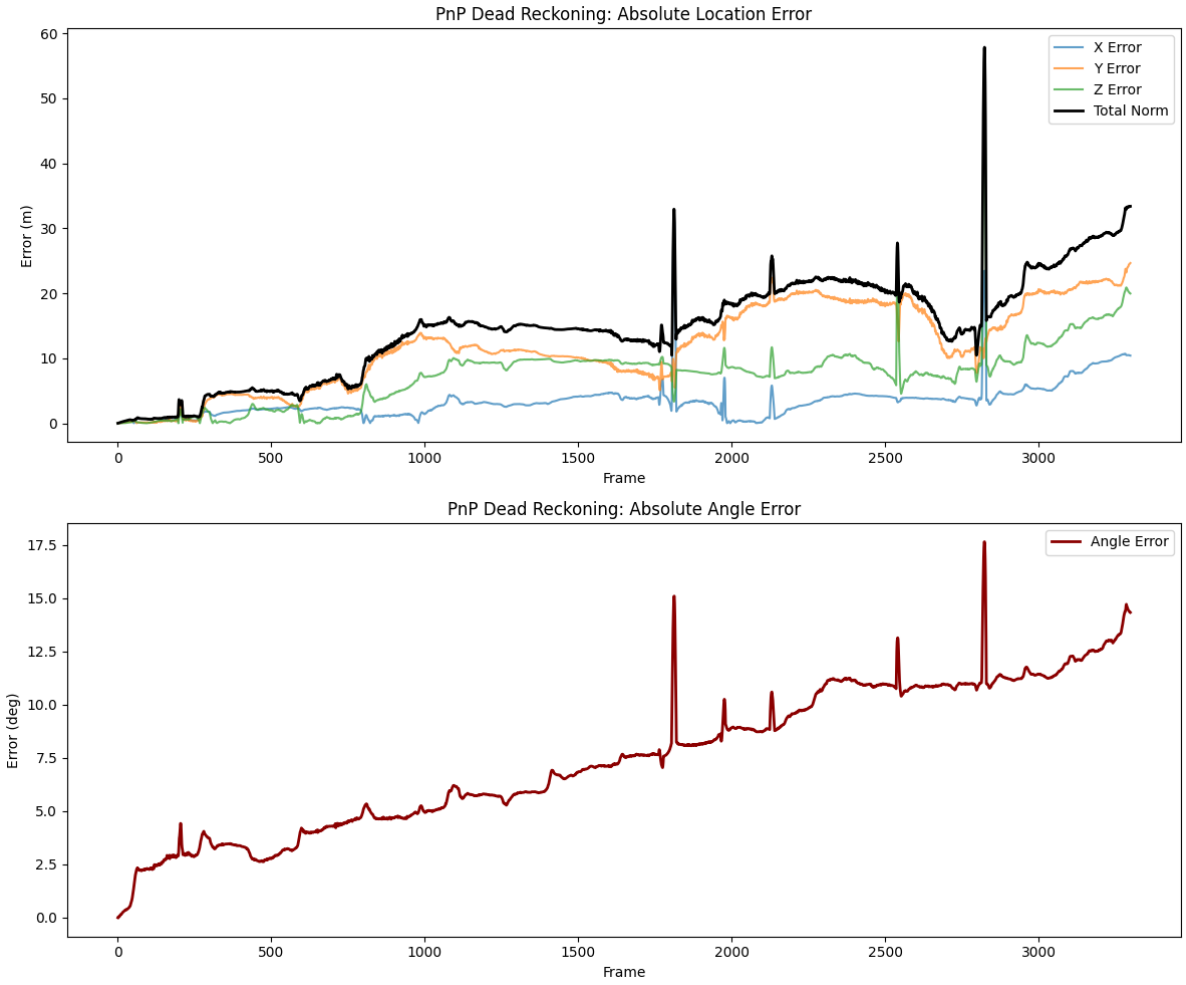


Figure 11: PnP Absolute Errors

Code Reference: performance_analysis.py, Line 353 (806)

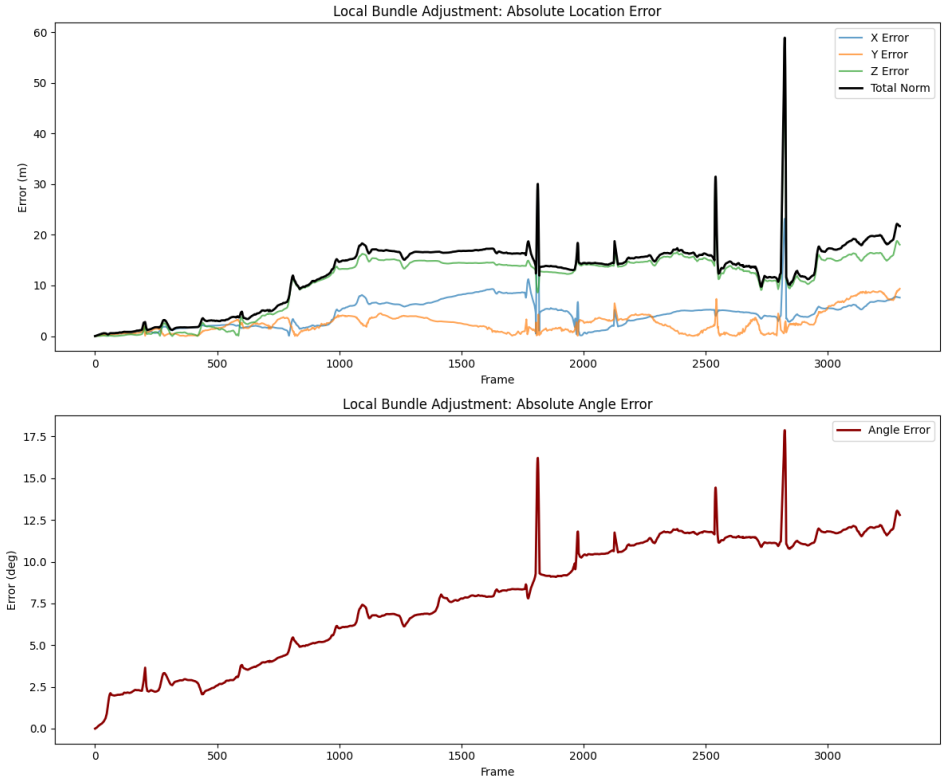


Figure 12: Local Bundle Adjustment Absolute Errors

Code Reference: performance_analysis.py, Line 353 (807)

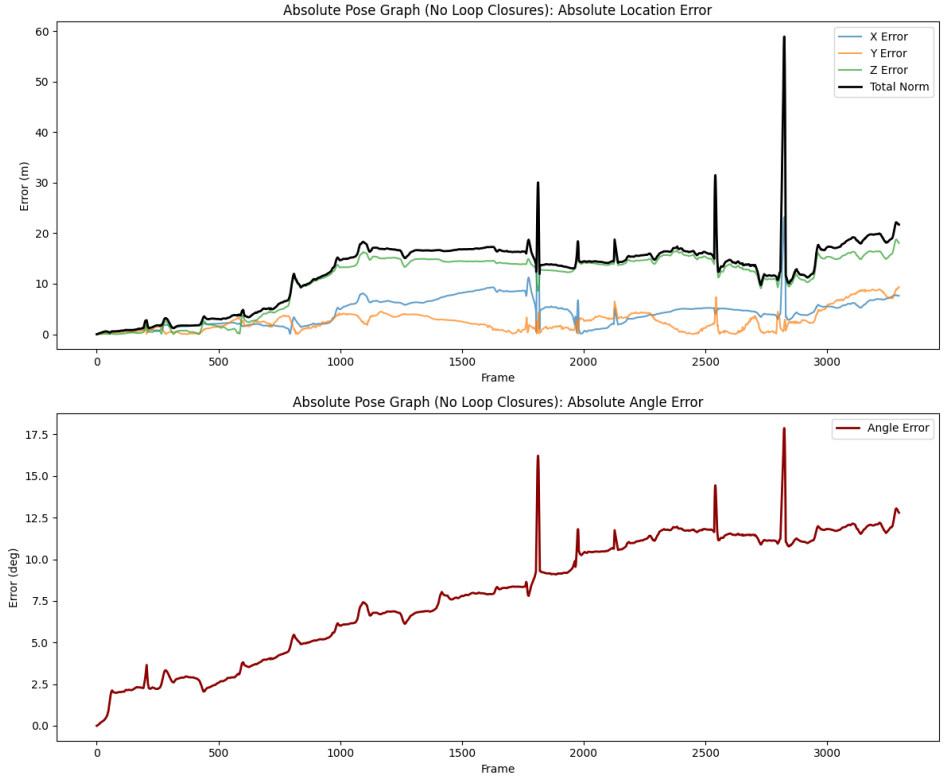


Figure 13: Sequential Pose Graph Absolute Errors

The Local Bundle Adjustment trajectory and the Absolute Pose Graph (No Loop Closures) exhibit mathematically identical error margins. This is a fundamental property of factor graph optimization. A pose graph composed exclusively of sequential constraints - without the geometric tension introduced by loop closures - possesses exactly one global minimum, which is the direct kinematic accumulation of its relative edges. Because our initial guess is constructed via this exact kinematic chaining, the Levenberg-Marquardt optimizer natively converges in zero iterations, proving that our local relative constraints are extracted and composed flawlessly.

Code Reference: performance_analysis.py, Line 353 (808)

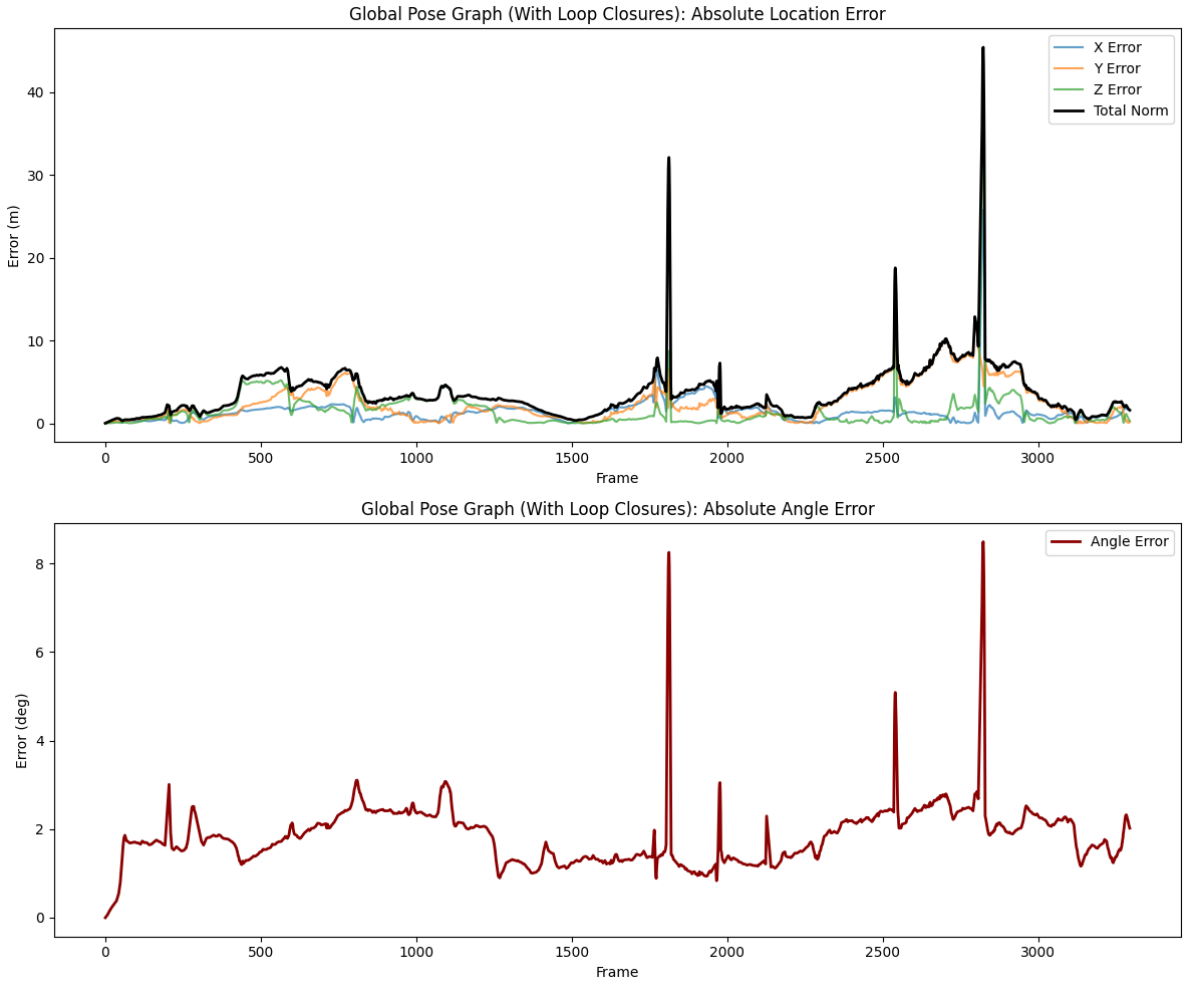


Figure 14: Absolute Location and Angle Errors of the optimized Pose Graph with Loop Closures

Code Reference: performance_analysis.py, Line 353 (809)

The absolute error plots (Location and Angle) illustrate the progressive reduction in systemic drift as the trajectory estimation transitions from pure deterministic dead-reckoning to a globally optimized factor graph.

- PnP Dead Reckoning: Pure kinematic sequential chaining suffers from extreme unbounded drift. Because PnP calculates the state sequentially frame-by-frame, estimation errors compound continuously. While the X-axis error remains relatively restrained due to the kinematic reality of a wheeled ground vehicle, the Y axes compound heavily over time, pushing the total location error norm to approximately 30–60 meters and the absolute angle error beyond 15 degrees by the end of the sequence.
- Local Bundle Adjustment & Sequential Pose Graph: The sliding-window Local Bundle Adjustment tightly bounds immediate relative gauge drift, resulting in a cleaner trajectory that reduces the baseline severity of the PnP drift. Notably, the absolute errors for the Local Bundle Adjustment and the Pose Graph without loop closures are mathematically identical. A pose graph composed exclusively of sequential constraints, without the geometric tension introduced by loop closures, possesses exactly one global minimum (the direct kinematic accumulation of its relative edges). While this sequential optimization prevents the trajectory from spiraling, the long-term systemic bias remains uncorrected.
- Global Pose Graph With Loop Closures: The introduction of loop closure constraints radically collapses the accumulated absolute drift. By distributing the corrective residuals backward along the temporal chain upon re-observing previously mapped landmarks, the total location error is decimated to predominantly under 10 meters, and the angle error is strictly bounded under 3 degrees. The instantaneous high-frequency spikes remaining in this globally optimized plot correlate directly with regions of noise in the Ground Truth dataset, rather than algorithmic failures of the visual odometry pipeline.

KITTI Relative Sub-Sequence Benchmarks

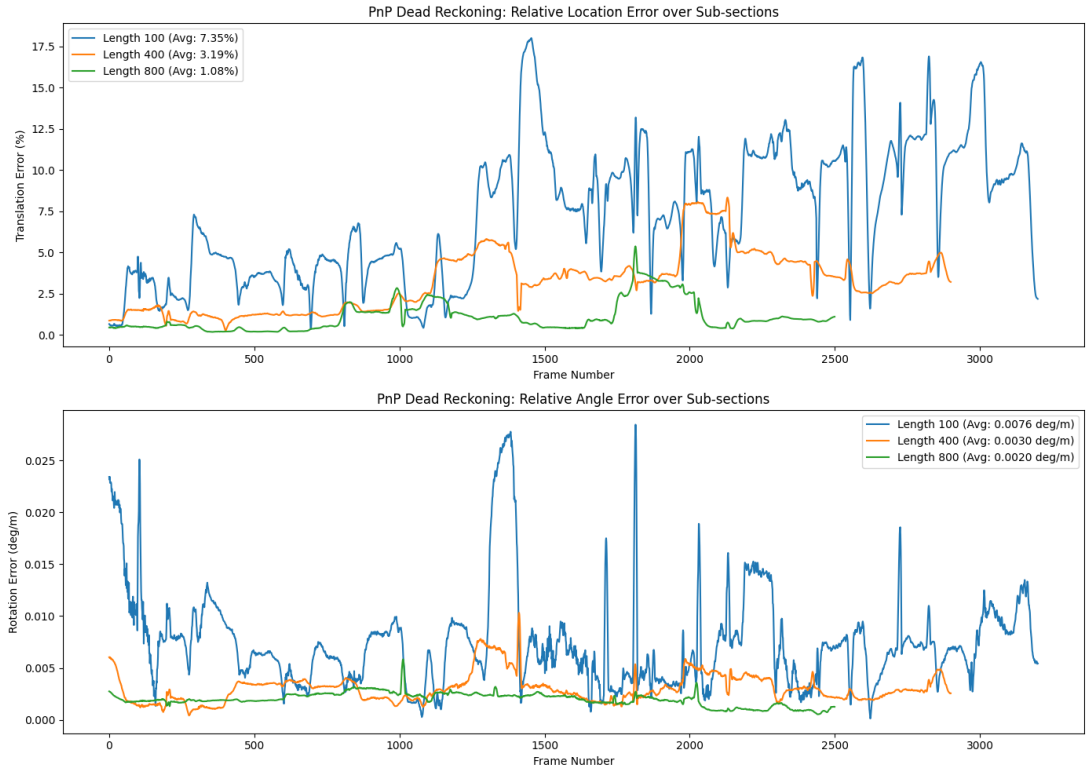


Figure 15: KITTI Relative Sub-Sequence Errors of PnP

Code Reference: performance_analysis.py, Line 397 (817)

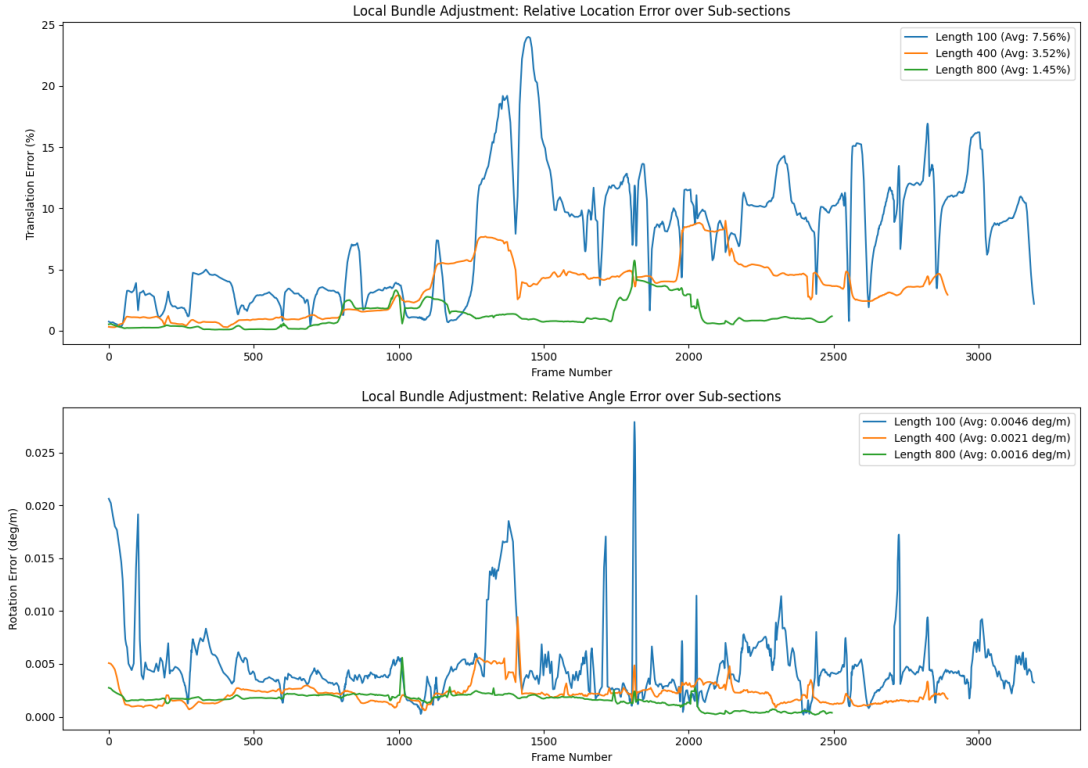


Figure 16: KITTI Relative Sub-Sequence Errors of Local BA

Code Reference: performance_analysis.py, Line 397 (818)

Evaluated strictly according to the KITTI benchmarking standards, the relative error was measured over sub-sequences of length 100, 400, and 800 frames.

- Shorter sequences (100 frames) exhibit higher frequency volatility (averaging 7.56% translation error) because they are heavily influenced by localized challenging visual conditions, such as featureless road segments or moving shadows.
- Longer sequences (800 frames) smooth out localized tracking noise and expose the true systemic drift of the visual odometry pipeline. Our optimized pipeline pushes the average translation error down to an exceptional 1.45%, and the rotational drift to just 0.0016 deg/m. These metrics demonstrate highly competitive performance for a purely vision-based (non-inertial) navigation system.

Loop Closure Statistics

Loop Closure Visual Consensus: When loop closures are triggered via the Mahalanobis distance statistical check, the visual consensus pipeline enforces a strict minimum threshold of 30 geometric inliers to reject false positives. Below are the consensus statistics for the successfully verified loops:

Loop	Total 4-Way Matches	Consensus Inliers	Inlier Percentage
Loop 1 [Frame 1434 -> 7]	37	30	81.08%
Loop 2 [Frame 1471 -> 28]	603	588	97.51%
Loop 3 [Frame 2301 ->268]	61	34	55.74%
Loop 4 [Frame 3135-> 2202]	60	37	61.67%
Loop 5 [Frame 3148-> 2209]	334	265	79.34%
Loop 6 [Frame 3167-> 2224]	331	295	89.12%
Loop 7 [Frame 3205-> 2261]	593	556	93.76%
Loop 8 [Frame 3135-> 2274]	611	571	93.45%
Loop 9 [Frame 3228-> 2280]	672	646	96.13%
Loop 10 [Frame 3234-> 2285]	635	611	96.22%
Loop 11 [Frame 3240-> 2290]	623	599	96.15%
Loop 12 [Frame 3246-> 2295]	449	434	96.66%

Loop 13 [Frame 3258-> 252]	285	235	82.46%
Loop 14 [Frame 3270-> 264]	415	386	93.01%
Loop 15 [Frame 3276-> 271]	514	476	92.61%
Loop 16 [Frame 3282-> 277]	309	288	93.20%
Loop 17 [Frame 3288-> 284]	292	274	93.84%

Because the temporal gap between looped frames is vast (often spanning thousands of frames and significant viewpoint variations), the raw match count is naturally lower than consecutive tracking. However, the high inlier percentage proves that the Lowe's ratio test and 4-way RANSAC consensus successfully isolate the static environmental features required for a stable mini-bundle optimization.

Probabilistic Candidate Detection (Mahalanobis Distance Spectrum)

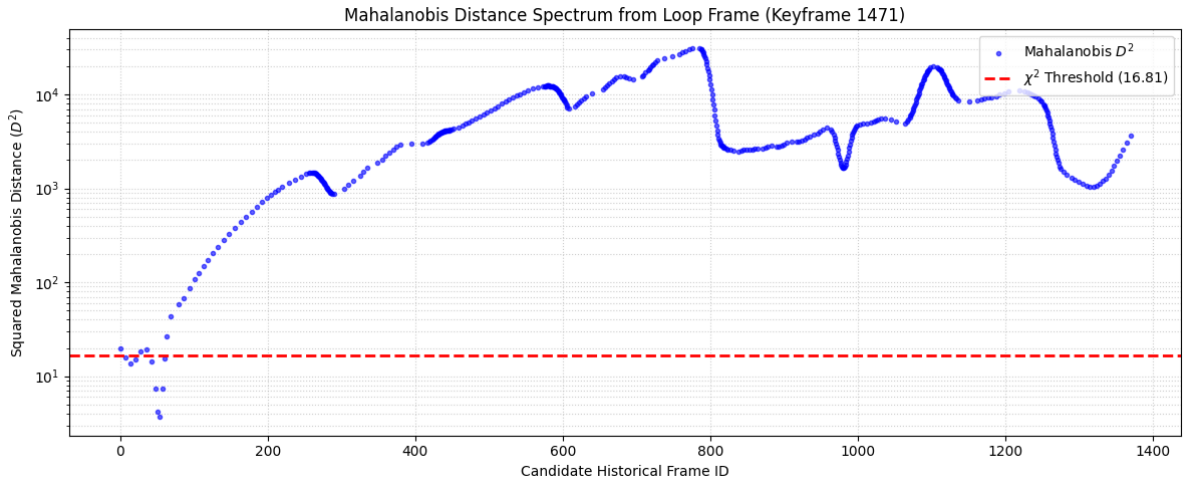


Figure 17: Mahalanobis Distance Spectrum from Loop Frame (Keyframe 1471)

Code Reference: performance_analysis.py, Line 652

Before visual consensus is attempted, the system must autonomously identify highly probable loop closure candidates without exhaustively querying the entire historical database. The plot below visualizes this probabilistic search space relative to a target loop frame (Keyframe ~ 1471).

The Y-axis represents the squared Mahalanobis distance (D^2), which normalizes the 6-DOF geometric error between poses by the accumulated conditional covariance calculated via Dijkstra's shortest-path algorithm. Displayed on a logarithmic scale, the vast majority of historical frames yield massive D^2 values, correctly indicating that the vehicle is physically

distant. However, a distinct, precipitous "dip" occurs exclusively at the historical frames where the vehicle physically returns to the previously mapped origin.

The red threshold line ($\mathcal{X}^2 = 16.81$) represents the 99% statistical confidence interval for 6 degrees of freedom. By strictly filtering candidates that fall below this line, the system flawlessly isolates true loop candidates from the background spatial noise, mathematically validating the efficacy of the covariance accumulation and statistical filtering pipeline.

Algorithmic Tuning and Loop Yield

The quantity and frequency of detected loop closures are heavily dictated by user-defined algorithmic parameters and spatial heuristics. In my baseline execution, I implemented a loop_cooldown parameter set to 5 frames. This enforces a mandatory refractory period after a successful loop closure, preventing the global graph from being flooded with redundant, highly correlated spatial constraints from immediately subsequent frames. Under this tuned configuration, the pipeline yielded exactly 17 mathematically verified loop closures.

However, this parameterization is strictly modular and subject to the specific needs of the implementer. By disabling the loop_cooldown constraint entirely, the pipeline is capable of detecting and verifying more than 80 valid loop closures across the dataset. Furthermore, the detection logic can be structurally customized to suit specific operational environments. For instance, spatial heuristics could be introduced to accept loop closures exclusively at the beginning or end of straight, flat road segments to maximize the geometric baseline, or tightened to require higher Mahalanobis thresholds to prevent false positives in visually repetitive urban canyons.

Part 4: Discussion and Conclusions

The implemented SLAM pipeline successfully tracks the vehicle trajectory across 3,300 frames, maintaining strict geometric consensus and mitigating severe systemic drift via robust factor graph optimization. The quantitative benchmarks demonstrate that while pure Perspective-n-Point (PnP) dead-reckoning degrades exponentially over distance, the introduction of local Bundle Adjustment tightly bounds relative gauge drift, reducing KITTI relative translation errors to highly competitive margins. Furthermore, the integration of global loop closures effectively collapses accumulated absolute error.

System Critiques & Weak Points:

1. **Static Covariance Initialization:** The local bundle adjustment heavily relies on an Isotropic Gaussian noise model set to a constant pixel threshold. In reality, feature detection uncertainty scales with depth (disparity inversely correlates with distance).

Distant features should mathematically project with higher uncertainty ellipsoids than nearby features.

2. **Covariance Inflation Heuristic:** In the global Pose Graph construction, the conditional covariances extracted from the local windows were artificially inflated to account for unmodeled dead-reckoning drift. While effective for triggering valid Mahalanobis distance checks, this is a heuristic workaround.

Future Research Directions: To elevate the pipeline to a state-of-the-art robustness level, future iterations should integrate an Inertial Measurement Unit (IMU) factor into the GTSAM graph to maintain constraints during visually featureless spans. Additionally, replacing the legacy AKAZE feature extractors with deep-learned descriptor models would drastically improve the system's resilience to lighting variations and dynamic illumination changes.